

Capture réelle du module Ventes

La vente est traitee comme une operation transactionnelle: controle metier, decrement du stock et creation.
Capture composee a partir des fichiers reels du depot GesPharmacie.

src/modules/vente/vente.routes.ts | lignes 8-13

```
8 | export const buildVenteRouter = (controller: VenteController) => {
9 |   const router = Router();
10 |   const validator = new VenteValidator();
11 |
12 |   router.get("/", asyncHandler(controller.list));
13 |   router.post("/", validate(validator.getCreateSchema()), asyncHandler(controller.create));
```

src/modules/vente/vente.service.ts | lignes 30-84

```
30 |   async create(data: CreateVenteDTO) {
31 |     return this.venteRepository.transaction(async (transaction) => {
32 |       const client = await this.venteRepository.findClientById(data.clientId, transaction);
33 |
34 |       if (!client) {
35 |         throw new AppError(404, "Client introuvable.", {
36 |           clientId: data.clientId,
37 |         });
38 |       }
39 |
40 |       const medicament = await this.venteRepository.findMedicamentById(
41 |         data.medicamentId,
42 |         transaction,
43 |       );
44 |
45 |       if (!medicament) {
46 |         throw new AppError(404, "Medicament introuvable.", {
47 |           medicamentId: data.medicamentId,
48 |         });
49 |       }
50 |
51 |       if (!isStrictlyAfterToday(medicament.dateExpiration)) {
52 |         throw new AppError(409, "Vente refusee: le medicament est expire.", {
53 |           medicamentId: data.medicamentId,
54 |           dateExpiration: medicament.dateExpiration,
55 |         });
56 |       }
57 |
58 |       if (medicament.qteStock < data.quantite) {
59 |         throw new AppError(409, "Stock insuffisant pour effectuer la vente.", {
60 |           medicamentId: data.medicamentId,
61 |           qteStock: medicament.qteStock,
62 |           quantiteDemandee: data.quantite,
63 |         });
64 |       }
65 |
66 |       const montantTotal = medicament.prix.mul(data.quantite);
67 |
68 |       await this.venteRepository.decrementMedicamentStock(
69 |         data.medicamentId,
70 |         data.quantite,
71 |         transaction,
72 |       );
73 |
74 |       return this.venteRepository.create(
75 |         {
76 |           clientId: data.clientId,
77 |           medicamentId: data.medicamentId,
78 |           quantite: data.quantite,
79 |           dateVente: data.dateVente,
80 |           montantTotal,
81 |         },
82 |         transaction,
83 |       );
84 |     });
```

src/modules/vente/vente.repository.ts | lignes 60-91

```
60 |   async decrementMedicamentStock(
61 |     id: string,
62 |     quantite: number,
63 |     transaction: Prisma.TransactionClient,
64 |   ): Promise<Medicament> {
65 |     return transaction.medicament.update({
66 |       where: {
67 |         id,
68 |       },
69 |       data: {
70 |         qteStock: {
71 |           decrement: quantite,
72 |         },
73 |       },
74 |     });
75 |   }
76 |
77 |   async create(
78 |     data: CreateVenteRecord,
79 |     transaction: Prisma.TransactionClient,
80 |   ): Promise<VenteDetails> {
81 |     return transaction.vente.create({
82 |       data,
83 |       include: {
84 |         client: true,
85 |         medicament: true,
86 |       },
87 |     });
88 |   }
89 |
90 |   async transaction<T>(callback: (transaction: Prisma.TransactionClient) => Promise<T>) {
91 |     return db.$transaction(callback);
```